Unit 6
SMD Practical + Oral Exam Notes

UNIT 6 — SOFTWARE ARCHITECTURAL DESIGN

Topic 1: Anatomy of Software Architecture

1. Simple Exam Definition:
Software Architecture is the high-level structure of a software system — defining its major components, how they interact, and the principles governing its design.

2. Core Concept Explanation:
Architecture answers 3 key questions:

- **What** are the major components?
- **How** do they communicate?
- **Why** are specific technologies/patterns chosen?

Key Architectural Concerns:

- **Structure:** How the system is organized (layers, modules, services).
- **Behaviour:** How components interact at runtime.
- **Quality:** Non-functional requirements (performance, security, scalability).

Topic 2: Quality Attributes (Non-Functional Requirements)

1. Simple Exam Definition:
Quality Attributes are non-functional requirements that define **how well** a system performs, not what it does. They are sometimes called "ilities."

2. Core Concept — Key Quality Attributes:

Attribute	Definition	SMS Example
Performance	Speed of response	Results load in < 2 seconds
Scalability	Handle increased load	10,000 students log in simultaneously
Availability	Uptime percentage	System is 99.9% available
Security	Protection against threats	Student A cannot see Student B's grades
Maintainability	Ease of change	Fix a bug without breaking other modules
Usability	Ease of use	A student can enroll in 3 clicks
Reliability	Consistent correct behavior	GPA calculation is always accurate
Testability	Ease of testing	Each layer can be tested independently

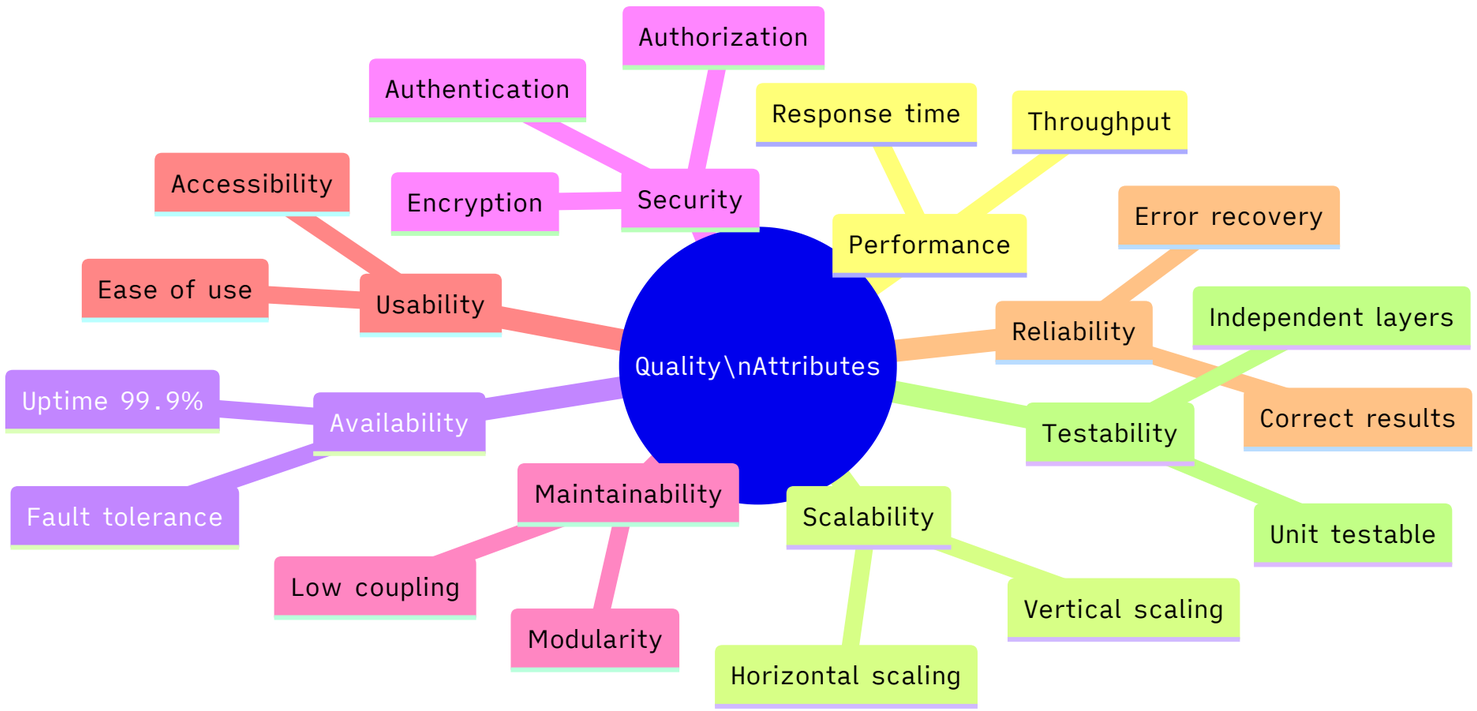
3. Examiner Viva Questions:

- What is the difference between functional and non-functional requirements?
- Name 5 quality attributes.
- Is 100% security achievable?

4. Strong Viva Answers:

- *Answer:* Functional requirements describe **what** the system does (login, enroll, grade). Non-functional requirements describe **how well** it does it (within 2 seconds, with 256-bit encryption).
- *Answer:* No, 100% security is not achievable or practical. Security is always a trade-off with usability and cost. We aim for "sufficient security" based on the risk level.

> 📊 **Visual: Quality Attributes — The "ilities"**



Topic 3: Client/Server Architecture

1. Simple Exam Definition:
Client/Server Architecture divides the system into two roles: **Clients** (request services) and **Servers** (provide services). Communication happens over a network.

2. Core Concept:

2-Tier (Classic Client/Server):

- Client talks directly to the database.

- Simple but problematic: Business logic is scattered between client and DB.
- Example: A desktop app directly querying MySQL.

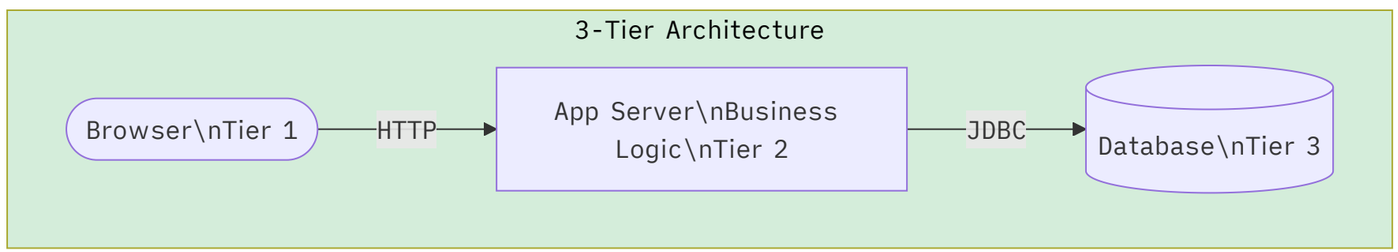
3-Tier Architecture (Most Common in Exams):

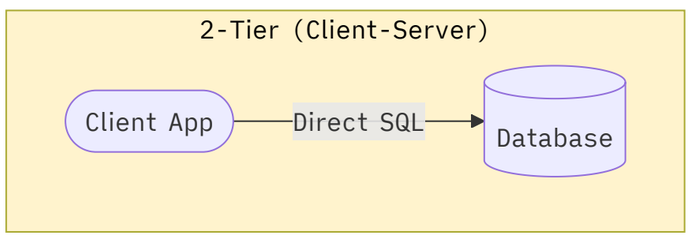
- **Tier 1 (Client):** UI — browser or app.
- **Tier 2 (Application Server):** Business Logic.
- **Tier 3 (Database Server):** Data Storage.
- Each tier is independent and can be scaled separately.

3. Real Practical Example (Student Management System):

- **Tier 1:** Student's web browser.
- **Tier 2:** Tomcat server running `StudentApp.war` (enrollment logic, grade calculation).
- **Tier 3:** MySQL database storing student records.

> 📊 **Visual: 2-Tier vs 3-Tier Architecture**





4. Examiner Viva Questions:

- What is the advantage of 3-tier over 2-tier?
- What is deployed on the Application Server in 3-tier?

5. Strong Viva Answers:

- *Answer:* In 3-tier, the business logic is on a separate server. This allows the database to be secured (not exposed to clients), business rules to be centralized, and each tier to be scaled independently.

Topic 4: Service-Oriented Architecture (SOA)

1. Simple Exam Definition:
SOA is an architectural style where functionality is exposed as **independent, reusable services** that communicate over a network using standardized protocols (REST, SOAP).

2. Core Concept:

- **Service:** A self-contained unit of functionality (e.g., `PaymentService`, `NotificationService`).
- **Service Contract:** Defines how to call the service (API endpoint, inputs, outputs).

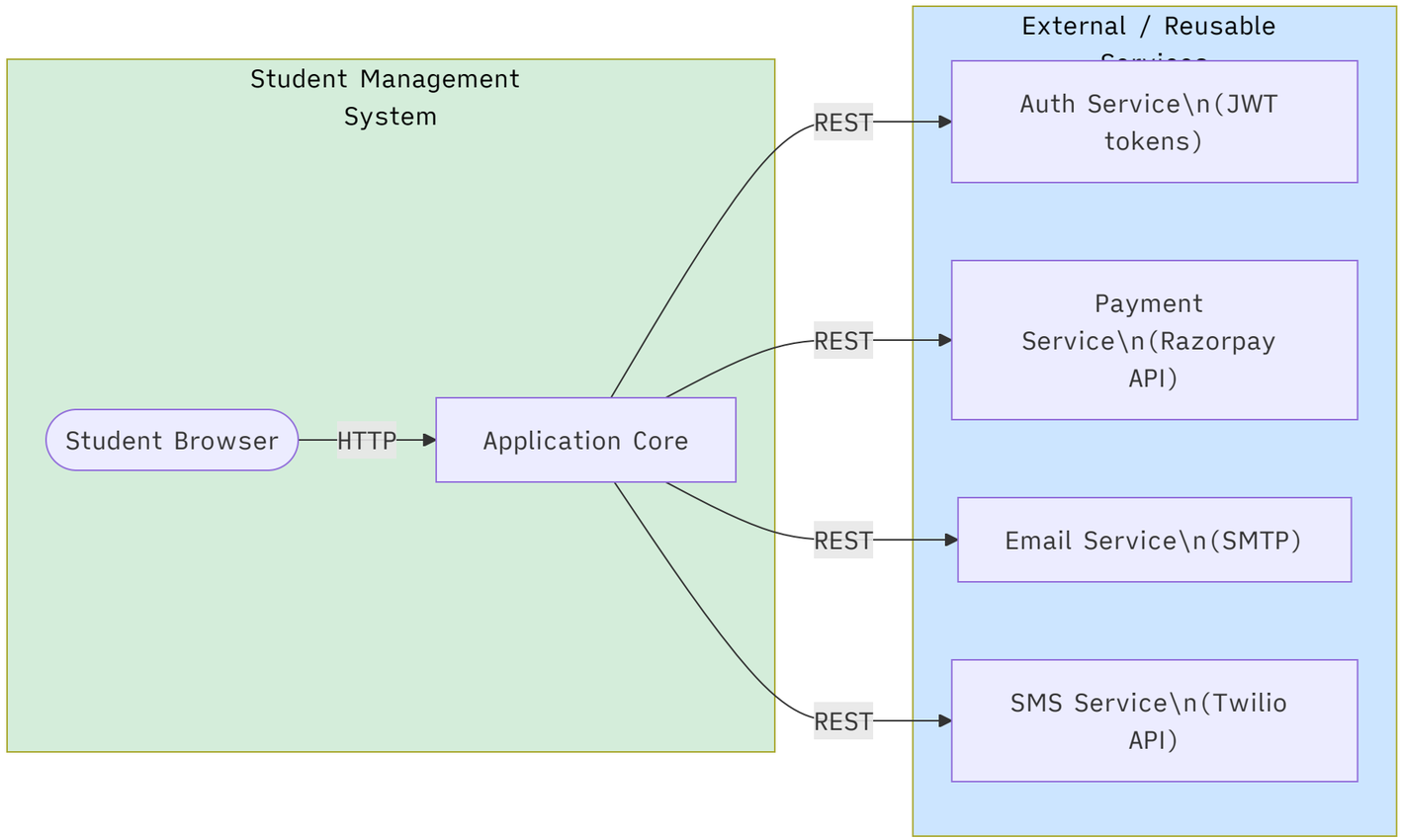
- **Loose Coupling:** Services are independent — changing one doesn't break others.
- **Reusability:** The same `EmailService` can be used by the Student System AND the Library System.

SOA vs Microservices: SOA services tend to be coarser-grained. Microservices are fine-grained services deployed independently. For this syllabus, focus on SOA.

3. Real Practical Example:
The Student Management System calls:

- `AuthService` for login.
- `PaymentGatewayService` for fee payment.
- `SMSNotificationService` for alerts.
  These are independent services that any system in the university can reuse.

> 📊 **Visual: SOA Architecture**



9. Difference Tables (C/S vs SOA):

Feature	Client/Server	SOA
Coupling	Tightly coupled	Loosely coupled
Communication	Direct / proprietary	REST, SOAP (standard)

Feature	Client/Server	SOA
Reusability	Low	High
Focus	Data serving	Service reuse

Topic 5: Component-Based Architecture

1. Simple Exam Definition:
Component-Based Architecture builds systems from **pre-built, reusable software components** — each with well-defined interfaces. Components are pluggable and replaceable.

2. Core Concept:

- **Component:** A self-contained, independently deployable software unit with a defined interface.
- **Interface:** The contract (methods) a component exposes.
- **Container:** The runtime environment hosting components (e.g., Java EE Application Server).

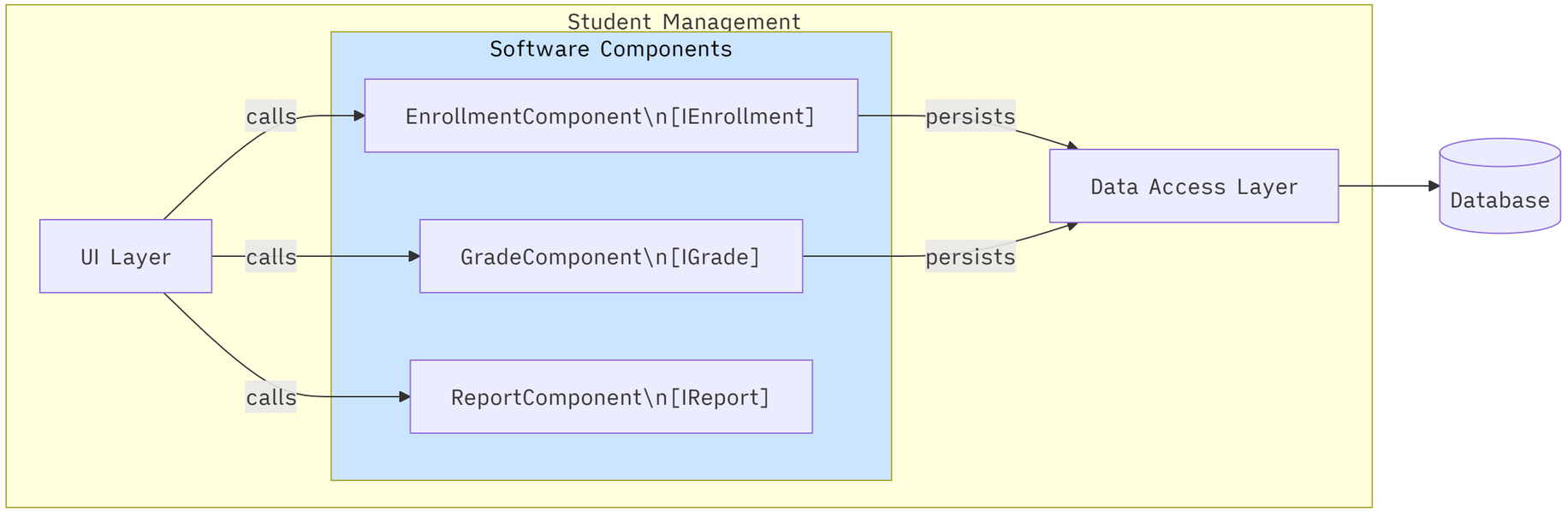
Key Principles:

- **Reusability:** A component is built once, used in many systems.
- **Replaceability:** One component can be swapped for another implementing the same interface.
- **Encapsulation:** Internal implementation is hidden.

3. Real Practical Example:

- `EnrollmentComponent` (handles enrollment logic).
- `GradeComponent` (handles grade calculation).
- `ReportComponent` (generates reports).
  Each is a separate, deployable module with a defined API.

> 📊 **Visual: Component-Based Architecture**



9. Difference Tables (SOA vs Component-Based):

Feature	SOA	Component-Based
Deployment	Network services (remote)	Local or remote components
Communication	REST/SOAP over network	Method calls or local IPC
Granularity	Coarse (whole service)	Fine (modular component)
Example	PaymentService over internet	GradeComponent in same app

---

Topic 6: Concurrent and Real-Time Architecture

1. Simple Exam Definition:
Concurrent Architecture designs systems where **multiple processes or threads execute simultaneously**. Real-Time Architecture adds the constraint that operations must complete within **strict time deadlines**.

2. Core Concept:

Concurrent Architecture:

- Multiple threads or processes run in parallel.
- Important for performance — e.g., multiple students enrolling simultaneously.
- Challenges: Race conditions, deadlocks, resource contention.

Real-Time Architecture:

- Hard Real-Time: Deadline MUST be met (e.g., airbag control in a car — if it fires 1 second late, it's useless).

- Soft Real-Time: Deadline is important but a small miss is acceptable (e.g., video streaming).

3. Real Practical Example:

- **Concurrent:** Enrollment server handles 5000 concurrent enrollment requests using a thread pool.
- **Real-Time (soft):** Student sees exam results within 500ms — if it takes 2 seconds, it's annoying but not catastrophic.

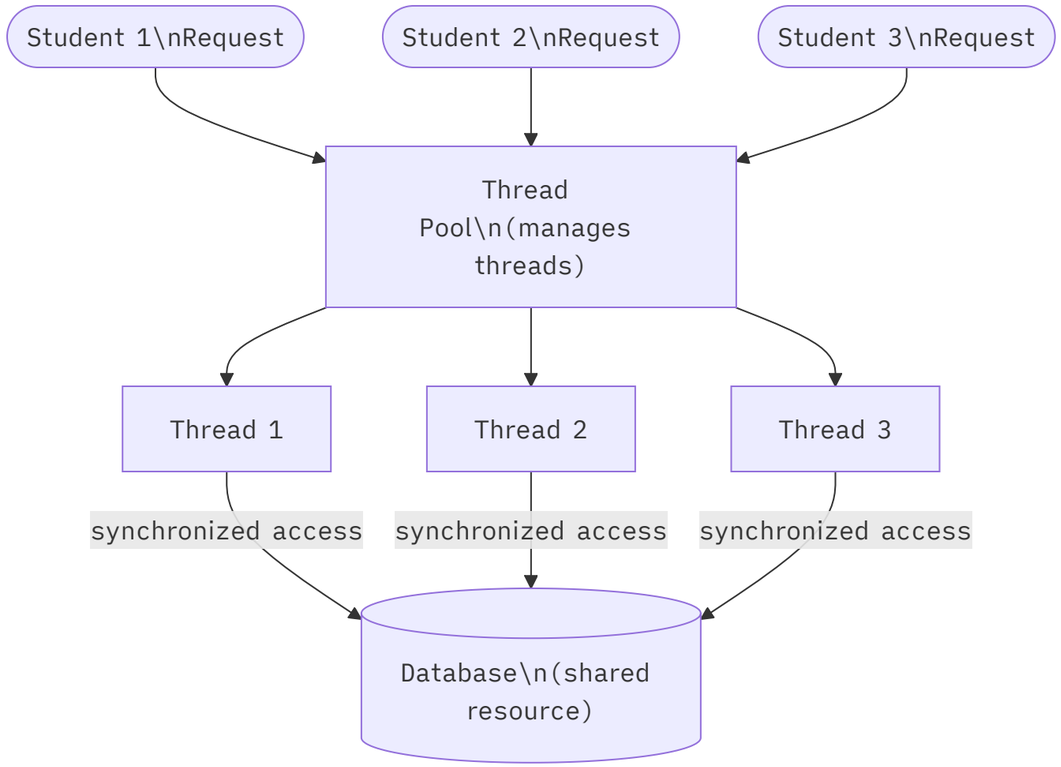
4. Examiner Viva Questions:

- What is the difference between concurrency and parallelism?
- What is a race condition?
- What is the difference between hard and soft real-time?

5. Strong Viva Answers:

- *Answer:* Concurrency means multiple tasks are in progress at the same time (may be interleaved on one CPU). Parallelism means tasks literally run simultaneously on multiple CPUs.
- *Answer:* A race condition occurs when two threads access shared data simultaneously and the outcome depends on the order of execution — this can cause incorrect results.

> 📊 **Visual: Concurrent Request Handling**



> 💡 **Exam tip:** When explaining concurrent architecture, mention that a **thread pool** is used to limit resource consumption (not creating a new thread per request), and **synchronized** blocks / **mutexes** prevent race conditions.

Topic 7: Product Line Architecture

1. Simple Exam Definition:
Product Line Architecture (Software Product Lines / SPL) is a set of **common architectural assets** shared across a family of related products, customized for each variant.

2. Core Concept:

- Build a **core platform** once with common features.
- Different **products** are derived from this platform with variable/optional features.
- Example: A university has one core SMS platform, but different colleges (Engineering, Medical, Arts) have customized portals — same architecture, different features.

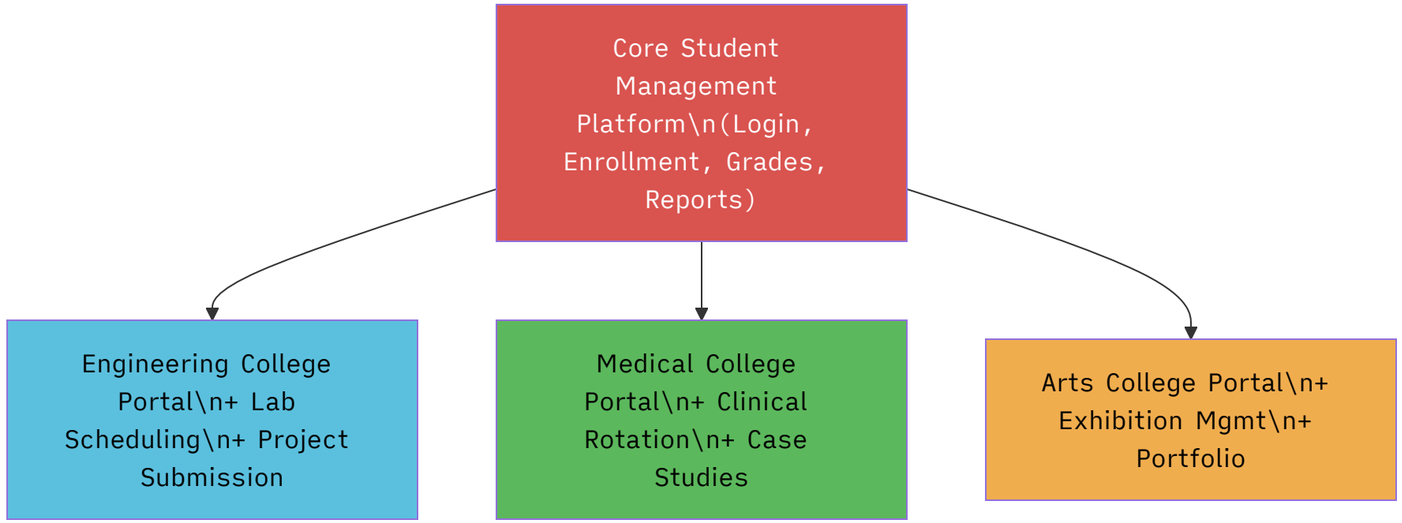
3. Key Concepts:

- **Core Assets:** Shared components, architecture, test cases.
- **Variability Point:** A place in the architecture where customization occurs.
- **Feature Model:** A diagram showing common and variable features.

4. Real Practical Example:

- Core: Login, Enrollment, Grades, Reports.
- Engineering College adds: Lab Scheduling, Project Submission.
- Medical College adds: Clinical Rotation Tracking, Patient Case Studies.

> 📊 **Visual: Product Line Architecture**



4. Examiner Viva Questions:

- What is a Software Product Line?
- What is a variability point?

5. Strong Viva Answers:

- *Answer:* A Software Product Line is a family of related systems built from a shared set of core assets. Different products customize the core assets at **variability points** to suit specific needs.

FINAL VIVA TIPS — PROFESSOR'S CORNER

> ⭐ **The Golden Rules for Maximum Marks:**

1. **Know your project cold.** Be ready to justify EVERY diagram you drew. "Why did you use Composition here?" → "Because the AcademicRecord cannot exist without the Student, sir."
2. **Arrow directions are everything.** External examiners specifically look for:
   - Hollow triangle = Generalization (points to parent).
   - Filled diamond = Composition (at the whole).
   - Hollow diamond = Aggregation (at the whole).
   - `<<include>>` arrow points TO the helper use case.
   - `<<extend>>` arrow points TO the base use case.
3. **Use the right vocabulary:**
   - Don't say "circle" — say "Initial Node" or "Actor."
   - Don't say "box" — say "Node" (deployment) or "Class" (class diagram).
4. **GRASP + GOF = Easy 5 marks.** Remember: "I used Singleton for DatabaseConnection because we don't want multiple connections, and Strategy for payment because we needed interchangeable payment methods."
5. **If you don't know something:** Say "Sir, I know this is related to [related concept], but I don't recall the exact term right now. Could I explain it from my project perspective?" — Never go silent.

> 📊 **Visual: Quick Revision — UML Arrow Cheat Sheet**

Association\n————————\nSo
line, no special end\nStuder
— Course

Generalization\n————————▷\nSo
line + hollow triangle\nStudent
——▷ User

Aggregation\n◇————————\nHollo
diamond at whole\nCourse ◇——
Syllabus

Composition\n◆————————\nFille
diamond at whole\nStudent ◆—
AcademicRecord

Depend
-
arrow\